```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <string.h>


#define MAX_OPS 100


typedef struct Node {
   int data;
        struct Node* left;
        struct Node* right;
} Node;


Node* createNode(int data) {
   Node *newNode = (Node*)malloc(sizeof(Node));
   newNode -> data = data;
        newNode -> left = NULL;
        newNode -> right = NULL;
        return newNode;
}


Node* insert(Node *root, int data) {
   if(root == NULL)
                return createNode(data);
        if(data < root -> data)
                root -> left = insert(root -> left, data);
        else if(data > root -> data)
```

```
                root -> right = insert(root -> right, data);
        return root;
}



Node* findMin(Node *root) {
   while(root && root -> left != NULL)
                root = root -> left;
        return root;
}



Node* deleteNode(Node *root, int key) {
   if(root == NULL)
                return NULL;
        if(key < root -> data)
                root -> left = deleteNode(root -> left, key);
        else if(key > root -> data)
                root -> right = deleteNode(root -> right, key);
        else{
                if(root -> left == NULL && root -> right == NULL){
                        free(root);
                                return NULL;
                }
                else if(root -> left == NULL){
                        Node *temp = root -> right;
                        free(root);
                        return temp;
                }
                else if(root -> right == NULL){
                        Node *temp = root -> left;
```

```
                        free(root);
                        return temp;
                }
                Node *temp = findMin(root -> right);
        root -> data = temp -> data;
        root -> right = deleteNode(root -> right, temp -> data);
        }
        return root;
}

// Search for a key in the BST.
bool search(Node *root, int key) {
    if(root == NULL)
                return false;
        if(key == root -> data)
                return true;
        else if(key < root -> data)
                return search(root -> left, key);
        else
                return search(root -> right, key);
}

int main() {
    int opCount;
    scanf("%d", &opCount);

    Node *root = NULL;

    char outputs[MAX_OPS][50];
    int outIndex = 0;
```

```
    int op, key;
    for (int i = 0; i < opCount; i++) {
        scanf("%d", &op);
        if(op == 1 || op == 2 || op == 3)
            scanf("%d", &key);

        switch(op) {
            case 1: // Insert
                root = insert(root, key);
                break;
            case 2: // Delete
                if(search(root, key))
                    root = deleteNode(root, key);
                break;
            case 3: { // Search
                if(search(root, key))
                    strcpy(outputs[outIndex++], "found");
                else
                    strcpy(outputs[outIndex++], "not found");
                break;
            }
            default:
                break;
        }
    }

    for (int i = 0; i < outIndex; i++) {
        printf("%s\n", outputs[i]);
}
            return 0;
}
```